

Assignment: Keys & Constraints — Hands-on & Scenario-Based Practice

Note: Work inside college_db.academics, building on the students table from previous assignments. Write and test every statement in pgAdmin's Query Tool — for scenario questions, actually run the steps described and observe PostgreSQL's real behavior before answering.

Setup Reference

Before starting, make sure the following base tables exist (create/adjust them as needed — you will be modifying them throughout this assignment):

Table	Purpose
academics.departments	Master list of departments (new — create in Q1)
academics.students	Student records (from previous assignments)
academics.courses	Course catalog (from last assignment)
academics.enrollments	Which student is enrolled in which course (new — junction table)

Q1. Design the departments Table

Create a departments table with dept_id as a Primary Key and dept_name that must be both NOT NULL and UNIQUE. Populate it with at least 4 departments.

```
-- Write your CREATE TABLE and INSERT statements here:
```

```
CREATE TABLE academics.departments
(
    dept_id SERIAL PRIMARY KEY,
    dept_name VARCHAR(100) NOT NULL UNIQUE
);

INSERT INTO academics.departments (dept_name)
VALUES
('Computer Science'),
('Information Technology'),
('Electronics'),
('Mechanical');
```

Q2. Retrofit a Foreign Key

Your students table currently has a department column of type VARCHAR — a design flaw, since it allows typos like 'Computr Science'. Add a new column dept_id (INTEGER) to students, and add a FOREIGN KEY constraint linking it to departments(dept_id). Do NOT drop the old department column yet.

```
-- Write your ALTER TABLE statements here:
ALTER TABLE academics.students
ADD COLUMN dept_id INT;
```

```
UPDATE academics.students
SET dept_id = 1
WHERE department = 'Computer Science';

UPDATE academics.students
SET dept_id = 2
WHERE department = 'Information Technology';

UPDATE academics.students
SET dept_id = 3
WHERE department = 'Electronics';

UPDATE academics.students
SET dept_id = 4
WHERE department = 'Mechanical';

ALTER TABLE academics.students
ADD CONSTRAINT fk_students_department
FOREIGN KEY (dept_id)
REFERENCES academics.departments(dept_id);
```

Q3. Composite Primary Key Junction Table

Create an enrollments table to model the many-to-many relationship between students and courses. It must have student_id and course_id as Foreign Keys, and together they should form a Composite Primary Key so a student cannot enroll in the same course twice. Also include enroll_date (DEFAULT to today) and grade (CHAR(2), nullable).

```
-- Write your CREATE TABLE statement here:
CREATE TABLE academics.enrollments
(
    student_id INT,
    course_id INT,
    enroll_date DATE DEFAULT CURRENT_DATE,
    grade CHAR(2),

    CONSTRAINT pk_enrollments
    PRIMARY KEY(student_id, course_id),

    CONSTRAINT fk_enrollment_student
    FOREIGN KEY(student_id)
    REFERENCES academics.students(student_id),

    CONSTRAINT fk_enrollment_course
    FOREIGN KEY(course_id)
    REFERENCES academics.courses(course_id)
);
```

Q4. Enforce a Valid Grade Range

Add a CHECK constraint on enrollments.grade so that only these values are allowed: 'A+', 'A', 'B+', 'B', 'C', 'F', or NULL (ungraded). Name the constraint explicitly.

```
-- Write your ALTER TABLE ... ADD CONSTRAINT statement here:
ALTER TABLE academics.enrollments
ADD CONSTRAINT chk_grade
CHECK
(
    grade IN ('A+', 'A', 'B+', 'B', 'C', 'F')
    OR grade IS NULL
);
```

Q5. CASCADE Delete Behavior

A student named 'Ravi Singh' withdraws from college entirely. The requirement is: if a student record is deleted from students, all of their rows in enrollments should be deleted automatically — no orphaned enrollment records should remain. (a) Write the FOREIGN KEY definition (or ALTER TABLE) needed on enrollments.student_id to achieve this. (b) Insert a test student and 2 enrollments for them, delete the student, then write a SELECT to prove the enrollments were removed automatically.

```
-- (a) Constraint definition:
ALTER TABLE academics.enrollments
DROP CONSTRAINT fk_enrollment_student;

ALTER TABLE academics.enrollments
ADD CONSTRAINT fk_enrollment_student
FOREIGN KEY(student_id)
REFERENCES academics.students(student_id)
ON DELETE CASCADE;

-- (b) Test INSERT, DELETE, and verification SELECT:
INSERT INTO academics.students
(student_id,name,email)
VALUES
(500,'Ravi Singh','ravi@test.com');

INSERT INTO academics.enrollments
(student_id,course_id)
VALUES
(500,1),
(500,2);

DELETE FROM academics.students
WHERE student_id=500;

SELECT *
FROM academics.enrollments
WHERE student_id=500;
```

Q6. RESTRICT vs CASCADE on courses

The college's policy is different for courses: a course must NEVER be deletable if students are still enrolled in it (to preserve academic records). (a) Define enrollments.course_id's foreign key with the correct ON DELETE behavior to enforce this. (b) Attempt to delete a course that has at least one enrollment and record the exact error PostgreSQL raises. (c) In 2-3 sentences, explain why this behavior protects data integrity better than CASCADE would here.

```
-- (a) Constraint definition:
ALTER TABLE academics.enrollments
DROP CONSTRAINT fk_enrollment_course;

ALTER TABLE academics.enrollments
ADD CONSTRAINT fk_enrollment_course
FOREIGN KEY(course_id)
REFERENCES academics.courses(course_id)
ON DELETE RESTRICT;

-- (b) DELETE attempt (paste the error message as a comment):
DELETE FROM academics.courses
WHERE course_id=1;
```

Q7. UNIQUE Constraint with a Real-World Twist

Two students can legitimately share the same name, but no two students should share the same email address. Add a UNIQUE constraint on students.email. Then attempt to insert a student with an email that already exists, and note what PostgreSQL does.

```
-- Write your ALTER TABLE and test INSERT statements here:
ALTER TABLE academics.students
ADD CONSTRAINT uq_student_email
UNIQUE(email);
```

Q8. SET NULL on Optional Relationship

Suppose faculty advisors are tracked in a new advisors table (advisor_id PRIMARY KEY, advisor_name), and students.advisor_id is a Foreign Key to it — but a student without an assigned advisor is still valid. If an advisor leaves the college and their row is deleted, existing students should NOT be deleted, but their advisor_id should become NULL. (a) Create the advisors table. (b) Add advisor_id to students with the correct ON DELETE behavior. (c) Test it: assign an advisor to a student, delete that advisor, and verify the student still exists with advisor_id now NULL.

```
-- (a), (b), (c) — write all statements here:
CREATE TABLE academics.advisors
(
    advisor_id SERIAL PRIMARY KEY,
    advisor_name VARCHAR(100)
);

ALTER TABLE academics.students
ADD COLUMN advisor_id INT;

ALTER TABLE academics.students
ADD CONSTRAINT fk_student_advisor
FOREIGN KEY(advisor_id)
REFERENCES academics.advisors(advisor_id)
ON DELETE SET NULL;
```

```
INSERT INTO academics.advisors(advisor_name)
VALUES('Dr. Sharma');
```

```
UPDATE academics.students
SET advisor_id=1
WHERE student_id=1;
```

```
DELETE FROM academics.advisors
WHERE advisor_id=1;
```

```
SELECT student_id,name,advisor_id
FROM academics.students
WHERE student_id=1;
```

Q9. Composite Unique Constraint

The college wants to ensure that within courses, no two courses in the same department can share the exact same course_name (duplicate course names ARE allowed across different departments). Add a constraint on courses that enforces uniqueness on the combination of (dept_id, course_name), not on course_name alone.

```
-- Write your ALTER TABLE ... ADD CONSTRAINT statement here:
ALTER TABLE academics.courses
ADD CONSTRAINT uq_course_department
UNIQUE(dept_id,course_name);
```

Q10. Debugging a Broken Insert

A junior developer ran the following statement and it failed. Diagnose why it failed based on the constraints you have built in this assignment, fix the underlying data or statement, and re-run it successfully. Document what was wrong.

```
INSERT INTO academics.students
(student_id,name,email)
VALUES
(9999,'Test Student','test9999@test.com');

INSERT INTO academics.enrollments
(student_id,course_id,grade)
VALUES
(9999,1,'A+');
```

```
-- Write your corrected/working statement(s) here:
```

Q11. CHECK Constraint with Multiple Conditions

Add a CHECK constraint to students that enforces: cgpa must be between 0 and 10 AND age (add this column if it doesn't exist, SMALLINT) must be at least 16. Use a single CHECK expression combining both conditions with AND.

```
-- Write your ALTER TABLE statement(s) here:
```

```
ALTER TABLE academics.students
ADD CONSTRAINT chk_student_age_cgpa
CHECK
(
    cgpa BETWEEN 0 AND 10
    AND
    age >= 16
);
```

Q12. Self-Referencing Foreign Key

The college wants to track reporting structure among staff. Using (or creating) an employees table with columns emp_id (PRIMARY KEY) and manager_id, add a Foreign Key on manager_id that references emp_id in the SAME table (a self-referencing relationship), allowing manager_id to be NULL for the top-level employee (e.g., the Principal). Insert at least 3 employees forming a two-level reporting chain and prove the relationship with a SELECT that shows each employee next to their manager's name (a self-join).

```
-- Write your CREATE/ALTER TABLE, INSERT, and self-join SELECT here:
```

```
CREATE TABLE academics.employees
(
    emp_id SERIAL PRIMARY KEY,
    emp_name VARCHAR(100),
    manager_id INT,

    CONSTRAINT fk_manager
    FOREIGN KEY(manager_id)
    REFERENCES academics.employees(emp_id)
);

INSERT INTO academics.employees
(emp_name,manager_id)
VALUES
('Principal',NULL);

INSERT INTO academics.employees
(emp_name,manager_id)
VALUES
('Vice Principal',1);

INSERT INTO academics.employees
(emp_name,manager_id)
VALUES
('Professor',2);

SELECT
e.emp_name AS employee,
m.emp_name AS manager
FROM academics.employees e
LEFT JOIN academics.employees m
ON e.manager_id = m.emp_id;
```

Note: Keep departments, students, courses, and enrollments populated with consistent data — Next (Joins & its Types) will query across all of these tables directly.